

```
while True: # Keep asking player until they enter a valid move.
    print(f"Player {playerTile}, enter 1 to {BOARD_WIDTH} or QUIT:")
    response = input("> ").upper().strip()

    if response == "QUIT":
        print("Thanks for playing!")
        sys.exit()
```

The function begins with an infinite loop that waits for the player to enter a valid move. This code resembles the `getPlayerMove()` function in the Tower of Hanoi program. Note that the `print()` call at the start of the `while` loop uses an f-string so we don't have to change the message if we update `BOARD_WIDTH`.

We check that the player's response is a column; if it isn't, the `continue` statement moves the execution back to the start of the loop to ask the player for a valid move:

```
if response not in COLUMN_LABELS:
    print(f"Enter a number from 1 to {BOARD_WIDTH}.")
    continue # Ask player again for their move.
```

We could have written this input validation condition as `not response.isdecimal() or spam < 1 or spam > BOARD_WIDTH`, but it's simpler to just use `response not in COLUMN_LABELS`.

Next, we need to find out which row a tile dropped in the player's selected column would land on:

```
columnIndex = int(response) - 1 # -1 for 0-based column indexes.

# If the column is full, ask for a move again:
if board[(columnIndex, 0)] != EMPTY_SPACE:
    print("That column is full, select another one.")
    continue # Ask player again for their move.
```

The board displays the column labels 1 to 7 onscreen. But the `(columnIndex, rowIndex)` indexes on the board use 0-based indexing, so they range from 0 to 6. To solve this discrepancy, we convert the string values '1' to '7' to the integer values 0 to 6.